# Denoising Diffusion Probabilistic Models for Driving Scenario Generation

Blake Taylor, Fang Fang, Inzaghi Moniaga

March 15, 2026

## I. Introduction

How can we train a self-driving car to anticipate the rarest and most dangerous driving scenarios? At an intersection, for example, a nearby vehicle may brake, accelerate, or turn, and each of these outcomes may be possible given the same observed history. A motion forecasting system, therefore, should not predict only a single most likely future; it should represent a distribution over possible futures, including rare but dangerous ones [Ettinger et al., 2021].

Traditional deterministic predictors are poorly suited to this setting, as human behaviour is inherently stochastic and trajectory prediction is fundamentally a probabilistic problem. Generative models offer a more appropriate alternative: rather than outputting one trajectory, they learn a distribution from which diverse and realistic futures can be sampled.

Among generative approaches, diffusion models have recently become especially successful in generating images [Kong et al., 2021, Chen et al., 2021], protein structure [Luo et al., 2022, Trippe et al., 2023, Ingraham et al., 2023], and driving scenario generation [Li et al., 2023, Chen et al., 2023, Wang et al., 2024], suggesting that they are a natural fit for realistic motion forecasting in autonomous driving.

In this work, we aim to investigate whether we can generate a realistic driving scenario using Diffusion Models by introducing denoising diffusion probabilistic models (DDPMs), applying the classifier-free guidance method [Ho and Salimans, 2021], and describe how these ideas can be applied to conditional motion forecasting on the Waymo Open Motion Dataset.

## II. Problem Setup

Given one-second pose histories of all vehicle agents on a provided map, the task is to simulate a realistic trajectory for all the agents.

Throughout the paper, $f = 1, \ldots, F$ indexes the $F$ future agent positions we wish to predict, and $t = 1, \ldots, T$ indexes the iterations of our denoising process. More specifically, for each predicted position $f$, we run $t = 1, \ldots, T$ denoising iterations.

### A. Forecast Goal

Our goal is to predict the future motion of all focal agents over a horizon of $F = 80$ timesteps. Let $\mathcal{C}$ denote the observed world context, which will be explained in more detail in Section D and let $\mathbf{x}_0$ represent the full future trajectory of the vehicle agent, defined in a local coordinate frame detailed below. Rather than outputting a single deterministic prediction, our objective is to model the conditional probability distribution over future trajectories to sample from:

$$p(\mathbf{x}_0 \mid \mathcal{C})$$

### B. Local agent coordinate system

If we train the model using global coordinates, it memorizes specific spots on the map. A left turn at one intersection uses completely different numbers than a left turn somewhere else. Instead, we shift everything to a local perspective. We place the car at the origin $(0, 0)$ and face it forward along the x-axis. Now, a left turn always involves the same embedding, regardless of the physical location. This allows the model to apply what it learns to entirely new maps.

To do this, let $\mathbf{p}_t \in \mathbb{R}^2$ be the focal agent's global position (acting as our *anchor*) and $\theta_t \in (-\pi, \pi]$ be its heading angle at the current time step $t$. Converting any arbitrary global coordinate $\mathbf{u}$ into this local frame involves two geometric steps:

1. **Translation:** We shift the entire world so that the focal agent sits exactly at the origin. We achieve this by subtracting the anchor position $\mathbf{p}_t$ from the target global coordinate $\mathbf{u}$.
2. **Rotation:** We rotate the world around this new origin so that the focal agent faces directly along the positive x-axis. We achieve this by applying a 2D

rotation matrix $\mathbf{R}$ evaluated at the negative anchor heading $-\theta_t$.

Mathematically, this local transformation is defined as:

$$\text{Local}(\mathbf{u}) = \mathbf{R}(-\theta_t)(\mathbf{u} - \mathbf{p}_t)$$

where the rotation matrix is given by:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

By applying this transformation to all inputs, every scenario is standardized relative to the focal agent's current position and heading. To visualize this process, Figure 1 illustrates how the global scene is reoriented around the focal agent.

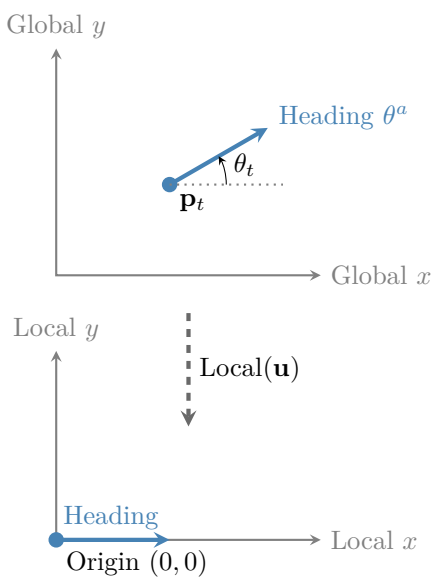


**Figure 1:** Geometric transformation from the global map coordinates to the agent-centric local reference frame.

### C. Trajectory representation

At each future step $f = 1, \ldots, F$, the model prediction target state is represented as

$$\mathbf{y}_f = (x_f,\, y_f,\, \sin\theta_f,\, \cos\theta_f) \in \mathbb{R}^4$$

where $(x_f, y_f)$ are local coordinates and $(\sin\theta_f, \cos\theta_f)$ encode heading without angular wrap-around. Rather than predicting the raw heading angle $\theta_f$ directly, the model predicts its sine and cosine components, which can be visualized in Figure 2.

In Figure 2 we see that predicting the angle directly introduces a severe mathematical discontinuity, as the

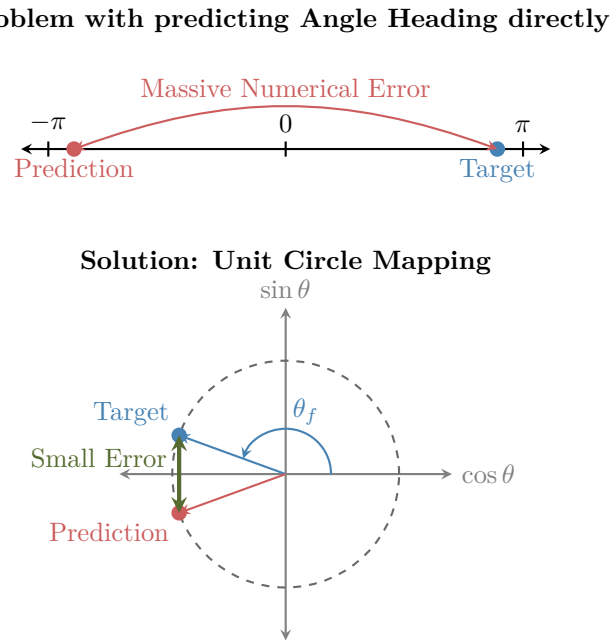


**Figure 2:** A neural network predicting raw angles is heavily penalized. Mapping the heading to a unit circle resolves this discontinuity and allows for smooth regression.

value abruptly jumps between $\pi$ and $-\pi$ when a vehicle completes a circle. By mapping the angle to a continuous point on the unit circle via $(\sin\theta_f, \cos\theta_f)$, we completely eliminate this wrap-around discontinuity. During inference, we can uniquely recover the predicted heading angle $\theta_f$ in the correct mathematical quadrant by applying the two-argument arctangent function, which can be visualized in Figure 2 as the angular sweep from the positive horizontal axis to the coordinate vector:

$$\theta_f = \text{atan2}(\sin\theta_f, \cos\theta_f)$$

Combining all future steps gives the clean target trajectory:

$$\mathbf{x}_0 = (\mathbf{y}_1, \ldots, \mathbf{y}_F) \in \mathbb{R}^{4F} = \mathbb{R}^{320}$$

This trajectory vector serves as the clean sample $\mathbf{x}_0$ used by the diffusion model.

### D. Scene conditioning representation

A vehicle does not drive on an empty map. To predict realistic futures, our model must understand the surrounding environment. We define this local scene description mathematically as a set of four distinct inputs:

$$\mathcal{C} = \{\mathbf{H}, \mathbf{N}, \mathbf{M}, \mathbf{S}\}$$

Each component captures a specific aspect of the driving environment:

- **History (H):** The focal agent's recent motion history. We look back over $H = 11$ past steps (including the current state) to capture its velocity, heading, and movement trends.
- **neighbours (N):** The spatial presence of nearby agents. We use $K = 6$ closest vehicles, pedestrians, or cyclists to establish interaction contexts and find the nearby neighbours.
- **Map (M):** Local road geometry represented as encoded polylines and points. This captures lane cues, traffic light conditions, and road edges to ensure the generated trajectory stays on the drivable surface and obeys structural rules.
- **Static Attributes (S):** Fixed properties like the agent's semantic type (vehicles, pedestrians, or cyclists) and the dimensional bounds of the object.

All these quantities are aligned to the local frame we defined earlier. To feed this complex data into our neural network, we process it through a `ConditionEncoder`. This network processes the historical trajectories (via GRU), applies cross-attention between agents and map features, and fuses the spatial, historical, map, and static attribute data into a single, dense vector:

$$\mathbf{c} = \text{ConditionEncoder}(\mathcal{C}) \in \mathbb{R}^{256}$$

This vector $\mathbf{c}$ summarizes the scene context.

# III. Background: Denoising Diffusion Probabilistic Models (DDPM)

Diffusion models are a class of generative models inspired by non-equilibrium thermodynamics [Weng, 2021]. The core intuition follows a familiar physical process: a drop of ink dispersing in water until its original structure is lost to diffusion. While reversing this dispersion is physically impossible, a neural network can be trained to learn the mathematical function required to undo this process. Consequently, a diffusion model operates via two distinct stages: the forward process and the reverse process.

## A. The Forward Process

The forward process is the simpler of the two stages. Given a clean data point $\mathbf{x}_0 \in \mathbb{R}^d$, where $d$ denotes the dimensionality of the data representation, we gradually destroy its structure by iteratively adding small amounts of Gaussian noise to create a sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$. This controlled corruption follows a recursive step:

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t \tag{III.1}$$

where $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is an independent standard Gaussian noise draw, and $\boldsymbol{\beta}_t \in (0, 1)$ is a scalar parameter dictating the variance schedule. The sequence $\beta_1, \beta_2, \ldots, \beta_T$ is predetermined, often linearly increasing, which means more noise is added as $t$ increases.

Since the update is a linear transformation of Gaussian noise, the conditional distribution is Gaussian with the following mean and variance:

$$\begin{aligned}
\mathbb{E}[\mathbf{x}_t \mid \mathbf{x}_{t-1}] &= \mathbb{E}[\sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t] \\
&= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathbb{E}[\boldsymbol{\epsilon}_t] && \text{(By Linearity)} \\
&= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \mathbf{0} && (\mathbb{E}[\boldsymbol{\epsilon}_t] = \mathbf{0}) \\
&= \sqrt{1-\beta_t}\mathbf{x}_{t-1}
\end{aligned} \tag{III.2}$$

Next, we calculate the conditional variance. Because $\mathbf{x}_{t-1}$ is treated as a given constant, its variance is strictly zero. When we pull the scalar multiplier of the noise term out of the variance operator, we must square it:

$$\begin{aligned}
\text{Var}(\mathbf{x}_t \mid \mathbf{x}_{t-1}) &= \text{Var}(\sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t) \\
&= \underbrace{\mathbf{0}}_{\text{Var(const)}=0} + \underbrace{(\sqrt{\beta_t})^2\text{Var}(\boldsymbol{\epsilon}_t)}_{\text{Var}(cX)=c^2\text{Var}(X)} \\
&= \beta_t\mathbf{I} \quad (\text{since Var}(\boldsymbol{\epsilon}_t) = \mathbf{I})
\end{aligned} \tag{III.3}$$

A fundamental statistical rule states that any linear transformation of a Gaussian variable results in another Gaussian variable. Therefore, $\mathbf{x}_t$ must be normally distributed with the exact mean and variance we just calculated. This gives us the formal transition probability:

$$p(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \;=\; \mathcal{N}\Big(\mathbf{x}_t;\; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\; \beta_t\mathbf{I}\Big) \tag{III.4}$$

A key property that makes training efficient is that we do not need to apply this step repeatedly to obtain $\mathbf{x}_t$ from $\mathbf{x}_0$. Because the sum of independent Gaussian variables remains Gaussian, all steps can be collapsed into a single closed-form expression. We introduce $\alpha_t = 1 - \beta_t$ to represent the signal retained at each step, allowing us to rewrite the state directly:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\bar{\boldsymbol{\epsilon}}_t \tag{III.5}$$

where $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$ is the cumulative product of the retained signal, and $\bar{\boldsymbol{\epsilon}}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the combined standard Gaussian noise resulting from the linear addition of all previous noise steps.

This gives us the probability of observing the noisy state $\mathbf{x}_t$ given only the original clean data $\mathbf{x}_0$:

$$p(\mathbf{x}_t \mid \mathbf{x}_0) \;=\; \mathcal{N}\big(\mathbf{x}_t;\; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\; (1-\bar{\alpha}_t)\mathbf{I}\big) \tag{III.6}$$

This formula has a clean geometric interpretation. The state $\mathbf{x}_t$ is a weighted mixture of the original signal and pure noise. Think of it as a cross-fade, where the first term represents how much of the original signal remains, and the second term fills in the rest with noise. By the final timestep $T$, $\bar{\alpha}_T \approx 0$ and all structures in the original data have been destroyed, leaving $\mathbf{x}_T$ as Gaussian noise.

### B. The Reverse Process

While the forward process is fixed and requires no learning, the reverse process must be learned. The goal is to train a neural network to approximate the reverse conditional distribution $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$, so that we can start from pure Gaussian noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively denoise back to a clean trajectory $\mathbf{x}_0$.

Directly computing the true reverse distribution $p(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is hard; we cannot guess where a noisy sample came from without more information. To get around this during training, we use the original clean data $\mathbf{x}_0$ as a guide. By applying Bayes Theorem to our known forward process and the clean starting point $\mathbf{x}_0$, we can map three interacting variables to transform an otherwise hard problem into a simple, easy-to-calculate Gaussian. First, let's review the Chain Rule of Probability:

$$p(A, B, C) = p(A \mid B, C)\, p(B \mid C)\, p(C) \tag{III.7}$$

Rewrite Equation (III.7) to isolate $p(A \mid B, C)$:

$$p(A \mid B, C) = \frac{p(A, B, C)}{p(B \mid C) p(C)} \tag{III.8}$$

By symmetry, we also know:

$$\begin{aligned} p(A, B, C) &= p(B, A, C) \\ &= p(B \mid A, C) p(A \mid C) p(C) \end{aligned} \tag{III.9}$$

Replacing the numerator, $p(A, B, C)$ from Equation (III.8) with Equation (III.9) gives:

$$\begin{aligned} p(A \mid B, C) &= \frac{p(B \mid A, C) p(A \mid C) p(C)}{p(B \mid C) p(C)} \\ &= \frac{p(B \mid A, C) p(A \mid C)}{p(B \mid C)} \end{aligned} \tag{III.10}$$

We can apply this exact probabilistic rule to our diffusion variables by setting $A = \mathbf{x}_{t-1}$, $B = \mathbf{x}_t$, and $C = \mathbf{x}_0$. This gives us the explicit form of our reverse process posterior:

$$\underbrace{p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)}_{\text{Reverse Process (posterior)}} = \frac{\overbrace{p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0)}^{\text{Forward Step (likelihood)}} \; \overbrace{p(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}^{\text{Prior}}}{\underbrace{p(\mathbf{x}_t \mid \mathbf{x}_0)}_{\text{Normalizing Constant}}} \tag{III.11}$$

We can simplify, since the forward diffusion process is a Markov chain. This means that once the current state $\mathbf{x}_{t-1}$ is known, the next state $\mathbf{x}_t$ no longer depends directly on the original clean sample $\mathbf{x}_0$:

$$p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) = p(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \tag{III.12}$$

So the posterior of Equation (III.11) becomes:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{p(\mathbf{x}_t \mid \mathbf{x}_{t-1})\, p(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{p(\mathbf{x}_t \mid \mathbf{x}_0)} \tag{III.13}$$

Next, notice that the denominator $p(\mathbf{x}_t \mid \mathbf{x}_0)$ does not depend on the variable we are solving for, namely $\mathbf{x}_{t-1}$. It only serves to normalize the distribution so that it integrates to one. Therefore, if we only care about the shape of the posterior as a function of $\mathbf{x}_{t-1}$, we can ignore this term as a constant of proportionality:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto p(\mathbf{x}_t \mid \mathbf{x}_{t-1})\, p(\mathbf{x}_{t-1} \mid \mathbf{x}_0). \tag{III.14}$$

Now we substitute the two Gaussian terms from the forward process:

$$\begin{aligned} p(\mathbf{x}_t \mid \mathbf{x}_{t-1}) &= \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\, \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \\ p(\mathbf{x}_{t-1} \mid \mathbf{x}_0) &= \mathcal{N}\Big(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\, \mathbf{x}_0, (1 - \bar{\alpha}_{t-1}) \mathbf{I}\Big) \end{aligned} \tag{III.15}$$

This gives:

$$\begin{aligned} p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto\; & \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\, \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \\ & \times \mathcal{N}\Big(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\, \mathbf{x}_0, (1 - \bar{\alpha}_{t-1}) \mathbf{I}\Big) \end{aligned} \tag{III.16}$$

At this point, the key observation is simple: both factors are Gaussians in the same unknown variable $\mathbf{x}_{t-1}$. The first term says that $\mathbf{x}_{t-1}$ should be a value that could plausibly produce the observed $\mathbf{x}_t$ in one forward noising step. The second says that $\mathbf{x}_{t-1}$ should also remain consistent with the clean sample $\mathbf{x}_0$. The posterior is therefore the compromise that satisfies both constraints at once.

To see this mathematically, we write out the exponents of both Gaussian distributions:

$$\begin{aligned} p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto \exp\Bigg( & -\frac{1}{2\beta_t} \left\| \mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1} \right\|^2 \\ & - \frac{1}{2(1 - \bar{\alpha}_{t-1})} \left\| \mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0 \right\|^2 \Bigg) \end{aligned} \tag{III.17}$$

Expanding the two quadratic terms and regrouping everything involving $\mathbf{x}_{t-1}$ produces a standard quadratic form. Completing the square reveals that this expression

is Gaussian in $\mathbf{x}_{t-1}$, from which the posterior mean and variance can be read off directly:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \sigma_t^2 \mathbf{I}) \tag{III.18}$$

The resulting mean and variance are

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \underbrace{\frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathbf{x}_0}_{\text{pull toward clean } \mathbf{x}_0} + \underbrace{\frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathbf{x}_t}_{\text{anchor to noisy } \mathbf{x}_t}$$
$$\sigma_t^2 = \underbrace{\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}}_{\text{scaled forward variance}} \tag{III.19}$$

This mean has a very natural interpretation: it is a weighted average of two pieces of information. One term pulls the sample back toward the clean trajectory $\mathbf{x}_0$, while the other keeps it close to the currently observed noisy state $\mathbf{x}_t$. Early in the reverse process, the model stays close to $\mathbf{x}_t$ and makes only small corrections. As the noise level decreases, the mean shifts more strongly toward $\mathbf{x}_0$.

### C. DDPM Inference Procedure

The difficulty during generation is that the clean sample $\mathbf{x}_0$ is unknown. Although the posterior distribution above is exact, we cannot evaluate it directly during inference.

DDPM resolves this limitation by predicting the noise added at step $t$ rather than directly predicting $\mathbf{x}_0$ or the posterior mean. Because the forward process has a closed form, we can rearrange Equation (III.5) to solve for $\mathbf{x}_0$:

$$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\bar{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}} \tag{III.20}$$

If a neural network can accurately predict the added noise $\boldsymbol{\epsilon}$ from $(\mathbf{x}_t, t)$, then it implicitly recovers $\mathbf{x}_0$ as well. Substituting this expression for $\mathbf{x}_0$ into the posterior mean removes the unknown clean sample entirely. This yields:

$$\tilde{\boldsymbol{\mu}}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) \tag{III.21}$$

This turns the reverse step into the following distribution:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}\big) \tag{III.22}$$

In other words, once the model can estimate how much noise is present in $\mathbf{x}_t$, it can take one denoising step backward.

**Algorithm 1** Inference Procedure

Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
**for** $t = T, \ldots, 1$ **do**
  Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
  $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
**end for**
**return** $\mathbf{x}_0$

Algorithm 1 translates the learned reverse process into a sequential generation loop. The procedure begins by drawing a pure noise sample from a standard Gaussian distribution. Inside the loop, the algorithm samples from the target Gaussian distribution using a shift and scale operation mathematically identical to the reparameterization trick [Kingma and Welling, 2013]. The procedure computes the exact deterministic mean via the neural network and completes the sample by adding standard Gaussian noise $\mathbf{z}$ scaled by the standard deviation $\sigma_t$. For the final step, we set the noise term to zero to ensure the output is the clean data point.

### D. DDPM Training Procedure

The training procedure is to minimize the objective function for the neural network to predict the noise $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ how close it matches the true injected noise $\boldsymbol{\epsilon}$. This is achieved using a simplified mean squared error objective:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}, t}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2\right] \tag{III.23}$$

In practice, the network parameters are updated via first-order stochastic optimization techniques such as AdamW, or Stochastic Gradient Descent.

**Algorithm 2** Training Procedure

**repeat**
  Sample clean trajectory $\mathbf{x}_0 \sim p(\mathbf{x}_0)$
  Sample timestep $t \sim \text{Uniform}(\{1, 2, \ldots, T\})$
  Sample Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  Take gradient step of Equation (III.23) w.r.t. $\theta$:
  $\nabla_\theta \left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\left(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t\right)\right\|^2$
**until** converged

Algorithm 2 describes the complete training procedure, where it begins by drawing a clean data point $\mathbf{x}_0$ from the true distribution and picking a random timestep $t$. After sampling a standard Gaussian noise vector $\boldsymbol{\epsilon}$, we generate a noisy version of the input. The neural network then predicts the specific noise vector applied to the data, and we take a gradient step with respect to $\theta$ to penalize any discrepancy between the actual noise and the predicted noise to improve the neural network weights.

# IV. Methods and Experiment

## A. Classifier-Free Guidance

The DDPM in Section II can generate realistic trajectories, but we also want those trajectories to match the scene context $\mathbf{c}$. Classifier-Free Guidance (CFG) [Ho and Salimans, 2021] achieves this by evaluating the same denoising network twice at the same noisy sample $\mathbf{x}_t$: once without context, and once with context.

Recall from Equation (III.5) that $\mathbf{x}_t$ is a noisy version of the clean trajectory $\mathbf{x}_0$. Given $(\mathbf{x}_t, t)$, the network produces

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c}_\emptyset) \quad \text{and} \quad \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c}),$$

where $\mathbf{c}_\emptyset$ is empty set for the context embedding. This implies that the first prediction is *unconditional*, while the second is *conditional* on the scene.

The key idea is that the difference between these two predictions captures how the scene should change the denoising direction. During sampling, we therefore use

$$\tilde{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, \mathbf{c}) = \underbrace{(1-w)\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing)}_{\text{suppressed unconditional score}} + \underbrace{w\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c})}_{\text{amplified conditional score}} \tag{IV.1}$$

where $w \geq 0$ is the guidance scale. If $w = 0$, generation is unconditional; if $w = 1$, we recover the usual conditional model; and if $w > 1$, the model follows the scene more strongly. In our experiments, we use $w = 1.2$.

For variance-preserving diffusion, combining noise predictions as in Equation (IV.1) is equivalent to combining scores, so CFG can be applied directly in the DDPM noise-prediction setting.

**A.0.1 Training.** To train one network to perform both unconditional and conditional denoising, we randomly drop the context during training:

$$\mathbf{c}' = \begin{cases} \mathbf{c}_\emptyset & \text{with probability } p_{\text{drop}}, \\ \mathbf{c} & \text{otherwise.} \end{cases} \tag{IV.2}$$

The training objective is the same noise-prediction loss as in Equation (III.23), except that the network receives $\mathbf{c}'$:

$$\mathcal{L}_{\text{cfg}} = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}, t}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \mathbf{c}')\right\|^2\right] \tag{IV.3}$$

**A.0.2 Inference.** At test time, each reverse step evaluates the network twice: once with $\mathbf{c}_\emptyset$ and once with $\mathbf{c}$. The two predictions are combined using Equation (IV.1), and the guided noise estimate $\tilde{\boldsymbol{\epsilon}}_\theta$ is used in the reverse DDPM update. Since this doubles the number of network evaluations per step, we sample with 50 reverse steps instead of the full $T = 200$.

## B. Trajectory Mapping or Decoding

Since diffusion-generated trajectories may still contain unrealistic motion artifacts, decoding strategies are introduced to convert model outputs into more physically plausible and safety-aware vehicle rollouts. We evaluate four progressive decoding models to bridge the gap between pure neural network outputs and strict vehicle kinematics.

### B.1 Model 1: Constant Velocity Baseline

The constant velocity baseline assumes that an agent continues to move with the exact velocity and heading observed at the current time. Let the initial position be $(x_0, y_0)$, the initial heading be $\psi_0$, and the initial speed be $v_0$. For a prediction horizon $T$ and time step $\Delta t$, the future states are extrapolated purely linearly:

$$\begin{aligned} x_{t+1} &= x_t + v_0 \cos(\psi_0)\Delta t \\ y_{t+1} &= y_t + v_0 \sin(\psi_0)\Delta t \\ \psi_{t+1} &= \psi_0 \\ v_{t+1} &= v_0 \end{aligned} \tag{IV.4}$$

This baseline is highly interpretable and serves as a fundamental reference point. While competitive for short-term straight-line motion, it completely ignores acceleration, turning behaviour, interactions with neighbouring agents, and road geometry. Therefore, it is generally inadequate for complex autonomous driving scenarios.

### B.2 Model 2: Raw Prediction

The legacy decoding model translates the diffusion output directly into global future states using a pure geometric transformation. Suppose the diffusion model predicts a local displacement $(\Delta x_t, \Delta y_t)$ and a heading representation $(\sin \Delta\psi_t, \cos \Delta\psi_t)$ for each future step $t$. We rotate this local displacement into the world frame using the anchor heading $\psi_0$:

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + \mathbf{R}(\psi_0) \begin{bmatrix} \Delta x_t \\ \Delta y_t \end{bmatrix} \tag{IV.5}$$

where $\mathbf{R}(\psi_0)$ is the standard two dimensional rotation matrix. The global heading is then reconstructed as:

$$\psi_t = \operatorname{atan2}(\sin \Delta\psi_t, \cos \Delta\psi_t) \tag{IV.6}$$

This approach is computationally efficient, but it treats each future state independently. Because it lacks a unifying kinematic framework, the resulting trajectory often

contains physically impossible lateral motion or jerky accelerations.

### B.3 Model 3: Bicycle Physics Baseline Model

To enforce temporal smoothness, we replace direct geometric decoding with a kinematic vehicle update. We interpret the diffusion model output not as absolute positions, but as motion proposals that supply the steering angle $\delta_t$ and acceleration $a_t$ [Polack et al., 2017]. We then generate the trajectory recursively using a simplified bicycle model with wheelbase $L$:

$$\begin{aligned} x_{t+1} &= x_t + v_t \cos(\psi_t)\Delta t \\ y_{t+1} &= y_t + v_t \sin(\psi_t)\Delta t \\ \psi_{t+1} &= \psi_t + \frac{v_t}{L}\tan(\delta_t)\Delta t \\ v_{t+1} &= v_t + a_t \Delta t \end{aligned} \tag{IV.7}$$

This approach eliminates implausible abrupt changes in position or orientation. However, because there are no explicit environmental constraints in this variant, the predicted vehicle may still drive off-road or collide with other agents.

### B.4 Model 4: Bicycle Physics Model With Safety Constraints

The full safety model uses the identical kinematic update equations defined above, but augments them with spatial and interaction-aware constraints. The generated states are iteratively corrected to ensure compatibility with the surrounding environment [Treiber et al., 2000].

First, a road-bound constraint discourages off-road motion by ensuring the distance to the nearest road edge $d_{\text{edge}}$ exceeds a minimum clearance threshold $d_{\min} > 0$:

$$d_{\text{edge}}(x_t, y_t) \geq d_{\min} \tag{IV.8}$$

Second, collision avoidance is enforced between any two agents $i$ and $j$ using a separation condition based on their respective safety radius $r_{ij}$:

$$\|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| \geq r_{ij} \tag{IV.9}$$

If these conditions are violated, a repulsion penalty is applied. Additional kinematic regularization, such as bounding the absolute speed $0 \leq v_t \leq v_{\max}$, guarantees that the trajectories are both physically plausible and globally safe.

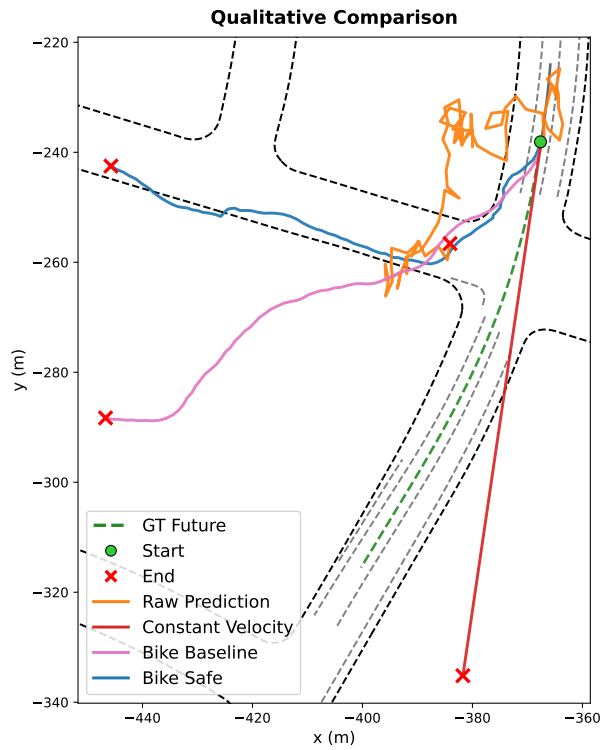


**Figure 3:** Comparison of four decoding strategies on Scenario 26. The safety-constrained bicycle decoder produces smoother trajectories and fewer off-road violations.

## V. Results and Discussion

To evaluate the performance of our models, we analyze two critical dimensions of the generated trajectories: positional accuracy and physical safety.

### A. Trajectory Accuracy

We first assess how close the predicted paths are to the actual paths taken by the agents. We measure this using Mean Displacement Error (MDE), which calculates the average Euclidean distance between the predicted and ground truth positions over a set number of future steps $T$.

$$MDE = \frac{1}{T}\sum_{t=1}^{T}\sqrt{(x_t^{pred} - x_t^{gt})^2 + (y_t^{pred} - y_t^{gt})^2} \tag{V.1}$$

Where:$(x_t^{pred}, y_t^{pred})$ represents the predicted position at time $t$ and $(x_t^{gt}, y_t^{gt})$ represents the ground truth position at time $t$

As shown in Figure 4, lower MDE values indicate more accurate predictions. The Constant Velocity (CV) baseline performs best overall. This is expected because the CV model assumes continuous speed and direction,

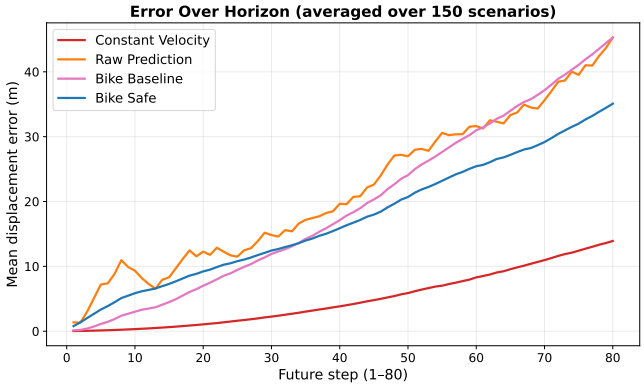


**Figure 4:** Mean Displacement Error over 150 scenarios

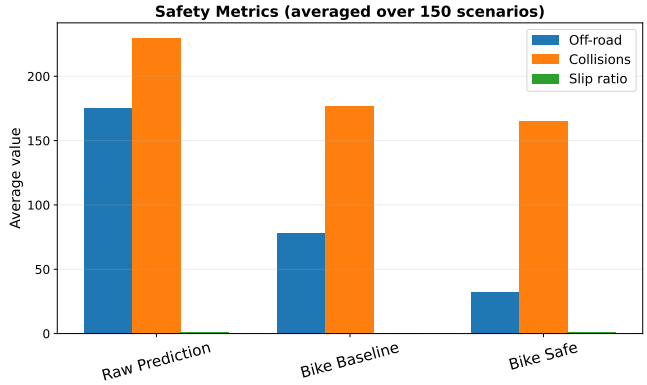


**Figure 5:** Safety metrics averaged over 150 scenarios with $d_{min} = 1.2$ meter, and $r_{ij} = 0.8$ meter

which perfectly matches the straight-line nature of most scenarios in our dataset.

When comparing the more complex models, the Legacy decoder and the Bicycle Physics model without safety constraints converge to nearly identical error rates by step 60. However, the Bicycle Physics model equipped with full safety constraints consistently outperforms both alternatives across the time horizon.

### B. Safety and Physical Realism

Accuracy alone does not guarantee a realistic or safe trajectory. Therefore, we also evaluate how each decoding strategy impacts physical realism using three post-processing safety metrics.

Let $M$ denote the set of evaluated models: the legacy decoder, the bicycle physics model without explicit safety constraints, and the full bicycle physics model with road-bound and interaction-aware corrections. For each model $m \in M$, we evaluate a safety profile defined by the tuple $\mathcal{S}(m) = (O(m), C(m), \Sigma(m))$:

- **Off-road violations** $O(m)$: Measures adherence to drivable space. An event is recorded if the distance to the nearest road edge falls below a clearance threshold $d_{\min} > 0$:

$$d_{\text{edge}}(\mathbf{p}_i(t)) < d_{\min}$$

- **Collision pairs** $C(m)$: Measures unsafe proximity between agents. A collision is registered if the distance between agents $i$ and $j$ violates a safety threshold $r_{ij}$ based on their respective sizes:

$$\|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| < r_{ij}$$

- **Mean slip ratio** $\Sigma(m)$: Quantifies lateral deviation from the forward heading. A smaller slip ratio indicates smoother and more physically realistic vehicle kinematics.

Figure 5 clearly shows that as we added more constraints to decoding the trajectory, we get a safer final result in terms of both off-road and collision.

## VI. Limitations

### A. CV baseline dominance.

The constant-velocity baseline outperforms all diffusion-based decoders in mean displacement error across the full 80-step horizon. This is expected given the straight-line majority of Waymo scenarios, but highlights that the diffusion model has not yet learned to leverage scene conditioning strongly enough to surpass a trivial predictor on average.

### B. Bicycle kinematics alone provide no benefit.

Figure 4 shows that the bicycle physics model without safety constraints converges to nearly identical error as the legacy decoder by step 60. This suggests that kinematic plausibility alone does not improve trajectory accuracy. The road-boundary enforcement and collision avoidance constraints are the critical ingredients that produce the reduction seen in the full model.

### C. Absence of external evaluation.

Results are evaluated using internally defined metrics, making it impossible to contextualize performance against published methods. The Waymo Scenario Generation Challenge provides a standardized Realism Meta Metric (RMM) that would allow direct comparison to results such as SimFormer (RMM = 0.6623).

## VII. Conclusion and Future Direction

In this project, we developed a context-conditioned diffusion framework for Waymo scenario generation, conditioning on agent history, neighboring agents, map geometry, and static object properties. The model predicts 80-step futures and supports both quantitative evaluation and qualitative visualization of rollouts.

Overall, results show strong short-horizon accuracy and generally realistic trajectories, especially when paired with lightweight post-processing to encourage safety and physical plausibility. Future work could improve scalability and realism by training on a larger portion of the dataset, extending to an autoregressive chunked forecasting scheme that predicts trajectories in segments up to a horizon $F$, and introducing additional generative components to increase diversity (e.g., a separate diffusion model for vehicle dimensions and randomized initial states).

## References


Chen Chen et al. Scenario dreamer: Vectorized latent diffusion for generating driving simulation environments. *arXiv preprint arXiv:2309.13110*, 2023.

Nanxin Chen, Yu Zhang, Heiga Zen, Ron J Weiss, Mohammad Norouzi, and Najim Dehak. Wavegrad: Estimating gradients for waveform generation. In *International Conference on Learning Representations*, 2021.

Scott Ettinger et al. Large scale interactive motion forecasting for autonomous driving: The waymo open motion dataset. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

Jonathan Ho and Tim Salimans. Classifier free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021.

John B Ingraham, Max Baranov, Zak Costello, Karl W Barber, Wujie Wang, Ahmed Ismail, Artem Lebedev, Alex Bennion, Pavel Golubkov, and Philip Bradley. Illuminating protein space with a programmable generative model. *Nature*, 2023.

Diederik P Kingma and Max Welling. Auto encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. Diffwave: A versatile diffusion model for audio synthesis. In *International Conference on Learning Representations*, 2021.

Mengxuan Li et al. Diffscene: Diffusion based safety critical scenario generation for autonomous vehicles. *arXiv preprint arXiv:2309.15103*, 2023.

Shitong Luo, Yufeng Su, Xingang Peng, Sheng Wang, Jian Peng, and Jianzhu Ma. Antigen specific antibody design and optimization with diffusion based generative models. *Advances in Neural Information Processing Systems*, 2022.

Philip Polack, Florent Altché, Brigitte d'Andréa Novel, and Arnaud de La Fortelle. The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles? In *2017 IEEE Intelligent Vehicles Symposium (IV)*, pages 812–818, 2017. doi: 10.1109/IVS.2017.7995816.

Martin Treiber, Ansgar Hennecke, and Dirk Helbing. Congested traffic states in empirical observations and microscopic simulations. *Physical Review E*, 62(2): 1805–1824, August 2000. ISSN 1095-3787. doi: 10.1103/physreve.62.1805. URL `http://dx.doi.org/10.1103/PhysRevE.62.1805`.

Brian L Trippe, Jason Yim, Doug Tischer, David Baker, Tamara Broderick, Regina Barzilay, and Tommi Jaakkola. Diffusion probabilistic modeling of protein backbones in 3d for the motif scaffolding problem. In *International Conference on Learning Representations*, 2023.

Yiyang Wang et al. Versatile behavior diffusion model for smart agent simulation. *arXiv preprint*, 2024.

Lilian Weng. What are diffusion models? *lilianweng.github.io*, Jul 2021.


## Gen AI declaration

Generate TikZ diagram, improve the plot readability, explain Code, debug, and write meaningful comment.

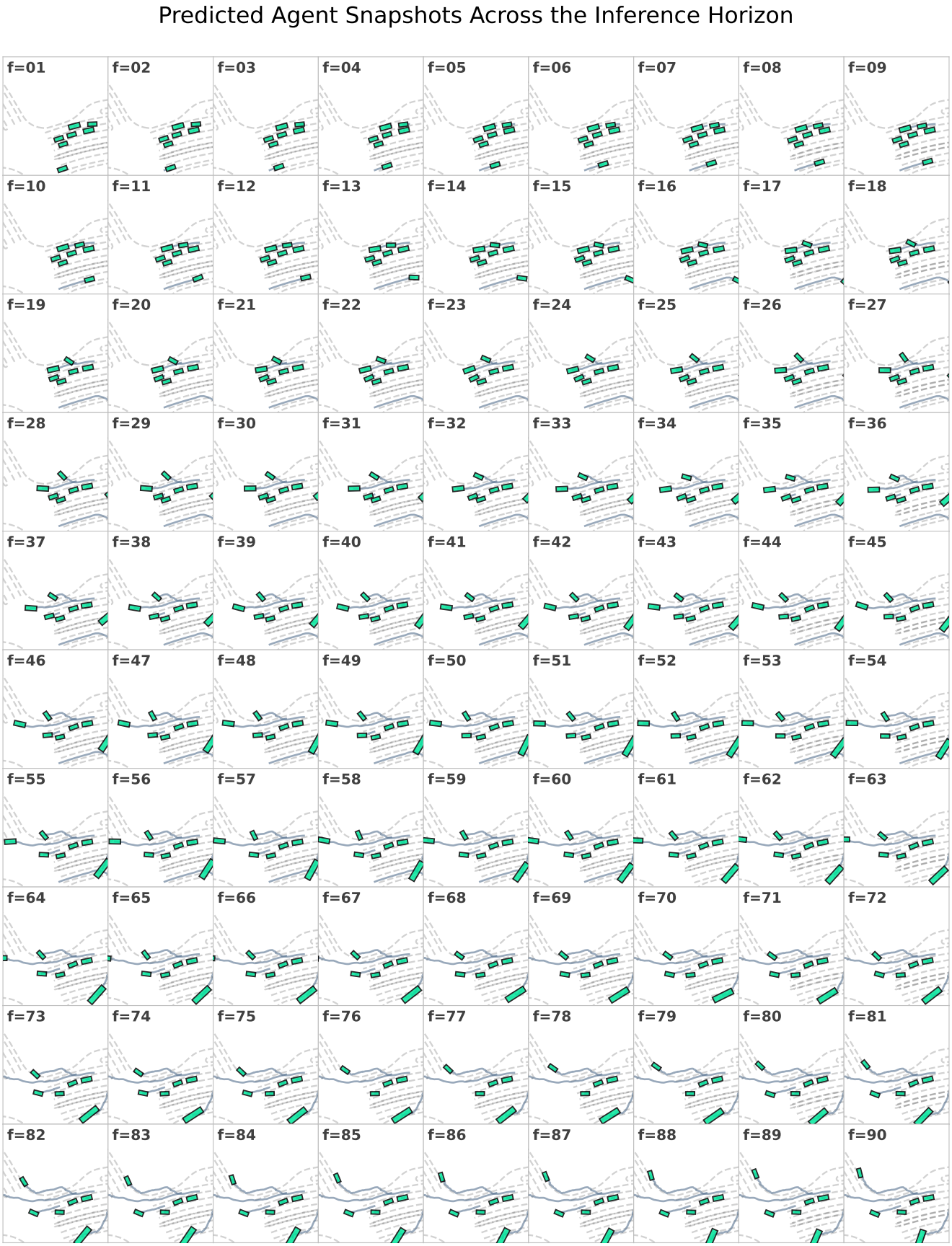


**Figure 6:** Appendix: 90 timesteps of Scenario 9